

LLM-Based Bug Detection and Automated Debugging Framework

Problem Statement

You are required to build a system that takes a GitHub repository URL as input, automatically analyzes the codebase, detects bugs, generates fixes using a Large Language Model (LLM), applies the fixes, and validates correctness by executing the system.

The framework must implement an end-to-end automated pipeline integrating static analysis, dynamic execution, LLM-based code repair, and validation.

Objectives and Requirements

1. Repository Processing

The system should automatically process a GitHub repository by:

- Accepting a GitHub repository URL as input.
- Cloning and analyzing the repository.
- Identifying:
 - Programming language(s).
 - Project structure.

2. Environment Setup and Execution

The framework should automatically prepare the execution environment by:

- Installing project dependencies (for example, requirements.txt, package.json).
- Handling environment configuration.
- Executing:
 - Unit tests (if available), or
 - The main application.

3. Bug Detection and Analysis

The framework should perform both static and dynamic analysis to detect software defects.

- Perform static analysis to detect:
 - Syntax errors.
 - Code smells.

- Perform dynamic analysis to detect:
 - Runtime errors.
 - Failing tests.

4. Bug Localization

The system should identify and localize detected bugs by:

- Mapping bugs to:
 - File(s).
 - Line number(s).
- Using:
 - Error logs.
 - Stack traces.
 - Static analysis outputs.

5. LLM-Based Code Repair

The framework should utilize a Large Language Model (LLM) to automatically generate fixes for detected bugs.

- The input to the LLM must include:
 - Buggy code snippet.
 - Error message or stack trace.
 - Additional context, if required.
- The output generated by the LLM must be:
 - Corrected source code, or
 - A software patch.
- Prompting strategies may include:
 - Few-shot prompting.
 - Chain-of-thought reasoning.

6. Patch Application

The framework should automatically apply the generated fixes by:

- Applying the generated patch to the codebase.
- Maintaining version changes (for example, diff/patch tracking).

7. Validation and Re-execution

After applying the generated fixes, the framework should:

- Re-run:
 - Unit tests, or
 - Program execution.
- Verify:
 - Whether the detected bug has been resolved.
 - Whether new issues have been introduced.

8. Iterative Feedback Loop

Implement a closed-loop automated debugging pipeline following the workflow:

Detect → Analyze → Fix → Apply → Execute → Validate → Repeat

9. Reporting and Logging

Generate a structured report containing:

- Detected bugs.
- Generated fixes.
- Execution results.
- Success or failure status.
- Before vs. After comparison.

Example components of the framework may include:

- Repository Handler
- Analysis Agent
- Bug Localization Agent
- Fix Generation Agent
- Execution and Validation Agent

Output Requirements

The completed framework should produce:

1. Detected bugs (with file and line references).
2. Generated fixes (code or patch).
3. Execution and validation results.
4. Iteration-wise logs (if an iterative loop is used).
5. Final system status (success or failure).
6. Summary report of changes.